

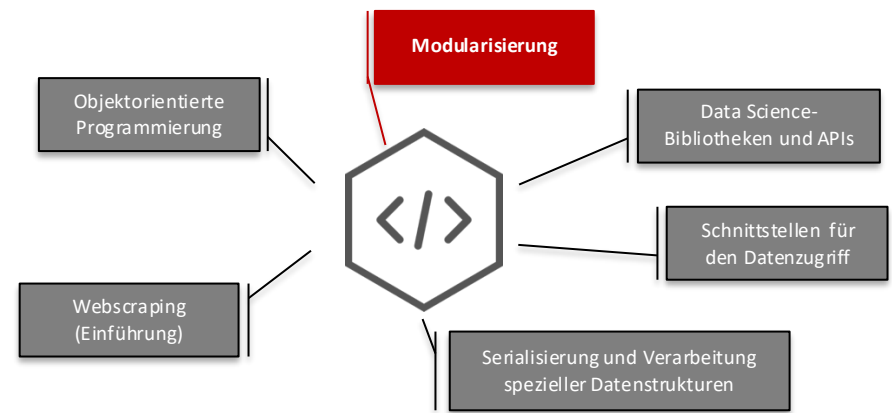
Lukas Weiß

DSCB230
Informatik für Data Science II

VL3 – Packages & Virtual Environments

Lernziele VL3

Packages und Virtual Environments



- Modularisierung in Python verstehen und eigene Pakete erstellen können
- Das Konzept der virtuellen Python-Umgebungen durchdringen und anwenden können
- Den Zusammenhang zu Virtualisierungsumgebungen nachvollziehen können
- Verständnis für Dependency Management

Agenda

■ Grundlagen der Modularisierung in Python

- ☐ Definition eigener Python-Packages
- ☐ Dependency Management
- ☐ Virtuelle Umgebungen (virtualenv & conda)
- ☐ Ausblick auf Containerisierung
- ☐ Zusammenfassung und Ausblick

Modularisierung – Grundlagen

- Motivation: Notwendigkeit zur Modularisierung
 - Wenn wir mehr und mehr Klassen in einer komplexen Applikation definieren, benötigen wir einen Weg, diese zu organisieren.
 - Beispiel: Das pandas Data Science-Paket
 - <https://github.com/pandas-dev/pandas>
 - Python-Module als erster Schritt zur Modularisierung und Strukturierung von Code
 - Module in Python sind „einfache“ Dateien (.py)
 - Im OOP-Kontext: Module enthalten die Klassen, die unser Software-Artefakt bilden.
 - Beispiel: <https://github.com/pandas-dev/pandas/blob/master/pandas/core/frame.py>
 - Wird ein Python-Programm in demselben Ordner ausgeführt, in dem auch ein Modul liegt, dann kann auf die Klassen und Methoden in dem Modul zugegriffen werden.
 - Wir importieren damit die Klassen und Methoden in den „namespace“ des laufenden Python-Programms.

Übung

- Programmieren Sie eine Klasse Point in einem Modul point.py.
 - Die Klasse soll einen Punkt im 2D-Raum repräsentieren und einige beispielhafte Methoden anbieten.

```
class Point:
    """ Point class represents and manipulates x,y coords. """

    def __init__(self, x=0, y=0):
        """ Create a new point at x, y """
        self.x = x
        self.y = y

    def distance_from_origin(self):
        """ Compute my distance from the origin """
        return ((self.x ** 2) + (self.y ** 2)) ** 0.5

    def __str__(self):
        return "({0}, {1})".format(self.x, self.y)

    @staticmethod
    def midpoint(p1, p2):
        """ Return the midpoint of points p1 and p2 """
        mx = (p1.x + p2.x)/2
        my = (p1.y + p2.y)/2
        return Point(mx, my)

    def halfway(self, target):
        """ Return the halfway point between myself and the target """
        mx = (self.x + target.x)/2
        my = (self.y + target.y)/2
        return Point(mx, my)
```

Modularisierung – Grundlagen

■ Import-Varianten

- Import einer bzw. mehrerer Klassen (oder Methoden) aus einem Modul

```
from modul import klasse, methode
```

zu vermeiden: `from modul import *`

- Import einer Klasse oder Methode aus einem Modul mit Alias

- Sinnvoll, wenn wir bspw. denselben Klassennamen bereits verwenden

```
from modul import klasse as kl
```

- Wir können das gesamte Modul auch importieren

- Dann erfolgt der Zugriff auf dessen Klassen aber mit `modul.klasse`

```
import modul
```

- Und auch hier sind Aliase möglich

```
import modul as md
```

Übung

- Programmieren Sie die beispielhafte Verwendung der Klasse Point in einem Modul `point_usage.py`.
 - Aufgabe: Anlegen von zwei Punkten und Ausprobieren der Klassenfunktionalitäten
 - Vorgehen: Probieren Sie verschiedene Import-Varianten aus

```
import point
p1 = point.Point(1, 1) # Zugriff auf Konstruktor
p2 = point.Point(2, 2) # Zugriff auf Konstruktor
midpoint = point.Point.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

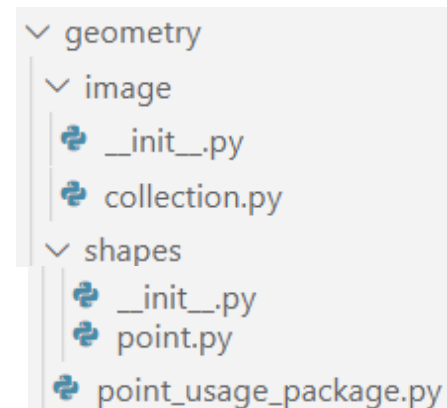
```
import point as pt
p1 = pt.Point(1, 1) # Zugriff auf Konstruktor
p2 = pt.Point(2, 2) # Zugriff auf Konstruktor
midpoint = pt.Point.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

```
from point import Point
p1 = Point(1, 1) # Zugriff auf Konstruktor
p2 = Point(2, 2) # Zugriff auf Konstruktor
midpoint = Point.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

```
from point import Point as Pt
p1 = Pt(1, 1) # Zugriff auf Konstruktor
p2 = Pt(2, 2) # Zugriff auf Konstruktor
midpoint = Pt.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

Modularisierung – Packages

- Herausforderung bei wachsendem Projektumfang
 - Es ist nicht sinnvoll, alle Module (also Python-Dateien) in nur einem einzigen Ordner abzulegen.
- Lösung in Python
 - Organisation der Module in Unterordner, die „packages“ genannt werden.
 - Der Name des packages entspricht dem Namen des Ordners.
 - Um einen Ordner in Python als package zu markieren genügt es, eine (zunächst leere) Datei `__init__.py` in den Ordner/Unterordner zu legen.
- Beispiel
 - geometry ist kein Package
 - image und shapes sind packages



Modularisierung – Packages

- Package-Imports können ebenfalls auf unterschiedliche Weise erfolgen
 - Package/Modul-Import
`import package.modul`
 - Import einer Klasse
`from package.modul import klasse, methode`
 - Modul-Import
`from package import modul`
- Bei Package-Hierarchien sind die Imports entsprechend anzupassen
 - Beispiel (alle Varianten von oben möglich):
`import package.subpackage.modul`

Übung

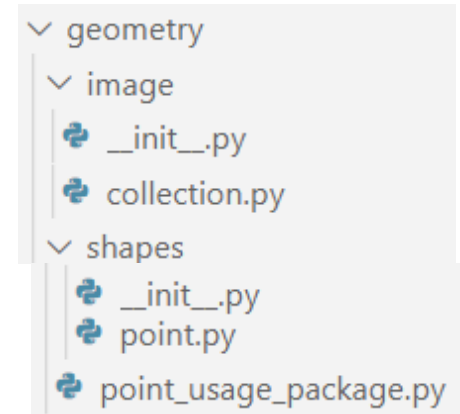
- Programmieren Sie die beispielhafte Verwendung der Klasse Point in einem Modul `point_usage_package.py`.
 - Arbeiten Sie im Unterordner `package_examples`
 - Jetzt liegt die Klasse in einer Package-Hierarchie
 - Vorgehen: Probieren Sie verschiedene Import-Varianten aus

```
import geometry.shapes.point
p1 = geometry.shapes.point.Point(1, 1) # Zugriff auf Konstruktor
p2 = geometry.shapes.point.Point(2, 2) # Zugriff auf Konstruktor
midpoint = geometry.shapes.point.Point.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

```
import geometry.shapes.point as pt
p1 = pt.Point(1, 1) # Zugriff auf Konstruktor
p2 = pt.Point(2, 2) # Zugriff auf Konstruktor
midpoint = pt.Point.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

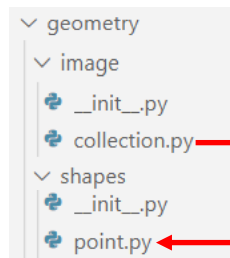
```
from geometry.shapes.point import Point
p1 = Point(1, 1) # Zugriff auf Konstruktor
p2 = Point(2, 2) # Zugriff auf Konstruktor
midpoint = Point.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```

```
from geometry.shapes.point import Point as Pt
p1 = Pt(1, 1) # Zugriff auf Konstruktor
p2 = Pt(2, 2) # Zugriff auf Konstruktor
midpoint = Pt.midpoint(p1, p2) # Zugriff auf statische Methoden
print(midpoint)
```



Modularisierung – Imports in SubPackages

- Sollen Module innerhalb der Klassen einer Package-Hierarchie untereinander verwendet werden, dann können die imports ggf. zu Fehlermeldungen führen.



Import von Klasse Point in Modul collection.py

```
from ..shapes.point import Point
```

Nicht empfohlen!

- Python erlaubt relative imports
 - Import eines Moduls auf selber Hierarchieebene

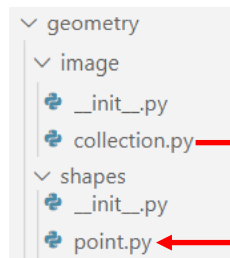
```
from .othermodule import klasse
```
 - Import in komplexen Hierarchien

```
from ..parentpackage.othersubpackage import modul
```

Modularisierung – Imports in SubPackages

- Empfehlung: Imports auf oberster Ebene starten lassen.

```
from parentpackage.othermodule import klasse
```



Import von Klasse Point in Modul collection.py

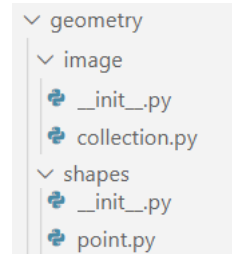
```
from geometry.shapes.point import Point
```

Empfohlen!

- In VSCode kann es allerdings je nach Ausführungsort zu import-Problemen führen (ModuleNotFoundError).
 - Dazu gibt es Workarounds und Einstellungsmöglichkeiten
 - Diese schauen wir uns nicht im Detail an.

Übung

- Programmieren Sie im Package „image“ / Modul `collection.py` eine Klasse `ShapeCollection`, die eine Liste von geometrischen Figuren pflegt.
 - Wir arbeiten zunächst nur mit Punkten. Welche Änderung sollten wir in der Klassenhierarchie ggf. vornehmen, um einen generischeren Ansatz zu erhalten?
- Darüber hinaus soll es eine Methode `add_random_points` geben, die zufällig positionierte Punkte der Liste von Shapes hinzufügt.
- Mittels der Methode `render_points` soll eine Bilddatei geschrieben werden, die die Punkte in der Liste als PNG-Datei (mit weißen Hintergrund) rendert.
 - Dabei sollen die Punkte zufällige Farben erhalten („red“, „green“, „blue“, „yellow“, „black“).
- Nutzen Sie die Methoden in einem Modul `collection_usage.py`
- Code: siehe GitHub



Agenda

- ☐ Grundlagen der Modularisierung in Python
- ☒ Definition eigener Python-Packages
- ☐ Dependency Management
- ☐ Virtuelle Umgebungen (virtualenv & conda)
- ☐ Ausblick auf Containerisierung
- ☐ Zusammenfassung und Ausblick

Definition und Publikation eigener Packages

- Neben der reinen Code-Struktur sind für eigene „hochwertige“ Packages weitere Aspekte zu berücksichtigen, bspw.
 - Definition, wie das Package „gebaut“ werden soll
 - Bspw. Nutzung von setuptools (<https://setuptools.readthedocs.io/en/latest/setuptools.html>)
 - Definition von Abhängigkeiten
 - Publikation des Quellcodes auf GitHub
 - Publikation in einem Package-Manager vorbereiten
 - „pip install mein-package“ (bspw. <https://pypi.org/>)
 - Befolgung von Standards bspw. bzgl. Code-Formatierung („pep8“)
 - Dokumentation des Packages
 - Inkl. automatischer Publikationsmöglichkeit (bspw. <https://www.sphinx-doc.org/en/master/>)
 - Informationen über Autoren, Lizenzen, Readme, ...
 - Bspw. <https://choosealicense.com/>
 - Tests – Qualitätssicherung des Quellcodes
 - Bspw. pytest (<https://docs.pytest.org/>) oder unittest (<https://docs.python.org/3/library/unittest.html>)

Übung

- Programmieren Sie ein Modul helloworld, das eine Methode `hello_world()` enthält. Diese soll mittels `print` „Hello“ ausgeben.
 - Legen Sie zuerst einen Ordner „helloworld“ an. Die Datei `helloworld.py` sollte dann in einem neuen Unterordner „src“ angelegt werden.
- Programmieren Sie in `setup.py` im Ordner helloworld Folgendes:

```
from setuptools import setup
```

```
setup(  
    name='helloworld',  
    version='0.0.1',  
    description='Say hello!',  
    py_modules=['helloworld'],  
    package_dir={'': 'src'}  
)
```

*Dies ist eine sehr einfache Basisdefinition
und kann von den „setuptools“ verwendet werden.*

- Navigieren Sie in der Kommandozeile in den Ordner helloworld und führen Sie `python setup.py bdist_wheel` aus.

Übung

- Bleiben Sie in dem Ordner helloworld. Dort sollten nun weitere Verzeichnisse angelegt worden sein.
- Nun können wir das Package „helloworld“ mittels pip installieren.
 - Achtung: wir nutzen jetzt eine lokale Installation.
 - Das Package ist nicht publiziert worden! *Dies werden wir auch nicht umsetzen.*
 - Geben Sie in der Kommandozeile ein: `pip install -e .`
 - Der Parameter „-e“ sorgt dafür, dass der Code verlinkt wird und wir weiter in dem Paket entwickeln könnten.
 - Der „.“ sorgt dafür, dass das Package im aktuellen Verzeichnis installiert wird.
- Das Package kann nun verwendet werden:

```
import helloworld  
helloworld.hello_world()
```

Hello

Übung

- Abhängigkeit können in der Datei setup.py hinterlegt werden.

```
from setuptools import setup

setup(
    name='helloworld',
    version='0.0.1',
    description='Say hello!',
    py_modules=['helloworld'],
    package_dir={'': 'src'},
    install_requires = [
        "pandas>=1.1.0"
    ]
)
```

- Führen Sie erneut „python setup.py bdist_wheel“ sowie „pip install -e .“ aus.
- Betrachten Sie die Ausgabe – pandas sollte installiert werden (bzw. die existierende Installation angezeigt werden).

Definition und Publikation eigener Packages

- Zur Publikation des Packages auf PyPi sind ein paar (wenige) weitere Schritte nötig. Wir gehen diesbezüglich nicht ins Detail.
 - Empfehlung: <https://www.youtube.com/watch?v=GIF3LaRqgXo>
- Die vorgestellte Definition und Struktur des Packages ist sehr rudimentär. Um die oben dargestellten Anforderungen umzusetzen, empfiehlt sich die Verwendung eines Tools wie poetry

Agenda

- ☐ Grundlagen der Modularisierung in Python
- ☐ Definition eigener Python-Packages
- ☒ **Dependency Management**
- ☐ Virtuelle Umgebungen (virtualenv & conda)
- ☐ Ausblick auf Containerisierung
- ☐ Zusammenfassung und Ausblick

- **Wollen wir immer ein eigenes Paket bauen?**



Dependency Management

- Abhilfe schafft die requirements.txt
 - Pinnt alle Abhängigkeiten auf eine getestete Version
 - Reproduzierbar auf jedem System
- pip-Dokumentation beschreibt das Requirements-File-Format am ausführlichsten
 - <https://pip.pypa.io/en/stable/reference/requirements-file-format/>
- [PEP 508](#) – definiert das Format für Dependency-Spezifikationen
- [PEP 440](#) – definiert das Versionsschema

Dependency Management

- Beispiele für Projekte mit requirements.txt
 - <https://github.com/odoo/odoo>
 - <https://github.com/pandas-dev/pandas>
 - <https://github.com/home-assistant/core>
 - <https://github.com/AUTOMATIC1111/stable-diffusion-webui>
 - <https://github.com/pytorch/pytorch>

Dependency Management

- Welches Problem entsteht?



Dependency Management

- Moderne Lösung: Poetry
 - <https://python-poetry.org/>
 - Werkzeug zur Abhängigkeitsverwaltung und Paketierung
- TOML Files
 - <https://de.wikipedia.org/wiki/TOML>
- Lockfile
 - Gewährleistung reproduzierbare Installationen

Dependency Management

```
[tool.poetry]
name = "lecture-task-app"
version = "0.1.0"
description = "Simple Flask task API for DSCB230"
package-mode = false
authors = ["Lukas Weiß <lweiss.weiss@exxeta.com>"]

[tool.poetry.dependencies]
python = "^3.12"
flask = "^3.1.3"
psycopg2-binary = "^2.9.12"

[tool.poetry.group.dev.dependencies]
ruff = "^0.15.12"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

Dependency Management



Übung

- Clone das github repo
- Installierendie Abhängigkeiten mit poetry
- Startet die Anwendung mittels poetry

Agenda

- ☐ Grundlagen der Modularisierung in Python
- ☐ Definition eigener Python-Packages
- ☐ Dependency Management
- ☒ Virtuelle Umgebungen (virtualenv & conda)
- ☐ Ausblick auf Containerisierung
- ☐ Zusammenfassung und Ausblick

Virtual Environments

- Python has its own unique way for downloading, storing, and resolving packages (i.e. modules that you can use within your Python code).
- Example
 - Open a Python interpreter
 - Import the package “numpy.random” (`import random`) and execute `random.randint(1, 10)`
 - This method returns a random number between 1 and 10.
 - But: where is this package located and why do we have access to it?
 - Enter `random` in the interpreter – there you’ll see the path to the package.

```
>>> random
<module 'random' from 'C:\\Users\\kupe0003\\AppData\\Local\\Programs\\Python\\Python39\\lib\\random.py'>
```

Virtual Environments

- Why are we talking about external packages and these details?
 - “It’s important to know this because, by default, every project on your system will use these same directories to store and retrieve site packages (third party libraries). At first glance, this may not seem like a big deal, and it isn’t really, for system packages (packages that are part of the standard Python library), but it does matter for site packages.” (Source: <https://realpython.com/python-virtual-environments-a-primer/>)
 - “Consider the following scenario where you have two projects: ProjectA and ProjectB, both of which have a dependency on the same library, ProjectC. The problem becomes apparent when we start requiring different versions of ProjectC. Maybe ProjectA needs v1.0.0, while ProjectB requires the newer v2.0.0, for example. This is a real problem for Python since it can’t differentiate between versions in the site-packages directory. So both v1.0.0 and v2.0.0 would reside in the same directory with the same name.” (Source: <https://realpython.com/python-virtual-environments-a-primer/>)
- ➔ This is where “virtual environments” or “conda environments” come into play...

Virtual Environments

- Basic purpose: create an isolated environment for Python projects.
 - Each project can have its own dependencies, regardless of what dependencies every other project has.
 - In the example above: create two separate virtual environments per project (ProjectA and ProjectB).
 - Each environment can depend on whatever version of ProjectC they choose.
 - There are no limits to the number of environments you can have
 - They're just directories containing a few scripts.
- Please note: Anaconda has a similar approach using “conda environments”

■ Creating a virtual environment

- (1) Let's find out where your Python installation is located.

- Open a Python interpreter and execute

```
import sys
sys.prefix
```
- In Windows, it should look similar to this:

```
>>> import sys
>>> sys.prefix
'C:\\Users\\kupe0003\\AppData\\Local\\Programs\\Python\\Python39'
>>> _
```

- (2) Navigate within a shell (cmd in windows) to the subfolder “Scripts”.

- There, you will find a program called pip.exe

```
C:\\Users\\kupe0003\\AppData\\Local\\Programs\\Python\\Python39\\Scripts>dir pip*
Volume in Laufwerk C: hat keine Bezeichnung.
Volumeseriennummer: 003E-6802

Verzeichnis von C:\\Users\\kupe0003\\AppData\\Local\\Programs\\Python\\Python39\\Scripts

15.04.2021  12:50           106.378 pip.exe ←
15.04.2021  12:50           106.378 pip3.9.exe
15.04.2021  12:50           106.378 pip3.exe
```

- pip is a package management system used to install and manage software packages written in Python.

■ Creating a virtual environment (cont'd)

■ (3) Upgrade pip and install virtualenv.

- `pip.exe install --upgrade pip`
`pip.exe install virtualenv`

- You may receive a warning that virtualenv is installed to a directory which is not part of the PATH variable. We'll ignore this warning.

■ (4) Create a virtual environment, e.g. within the tmp-folder, using `virtualenv c:\tmp\venvs\venv1`

```
C:\tmp\venvs>virtualenv venv1
created virtual environment CPython3.9.1.final.0-64 in 3306ms
  creator CPython3Windows(dest=C:\tmp\venvs\venv1, clear=False, no_vcs_ignore=False, global=False)
  seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=C:\Users\kupe06\AppData\Local\pypa\virtualenv)
    added seed packages: pip==21.0.1, setuptools==54.1.2, wheel==0.36.2
  activators BashActivator,BatchActivator,FishActivator,PowerShellActivator,PythonActivator,XonshActivator
```

- A new folder should be generated (venv1).

Virtual Environments

Please note: The exercises won't work if your only Python installation is based on Anaconda!

■ Creating a virtual environment (cont'd)

- Fifth, activate this virtual environment as the environment you want to work with for now.
 - `C:\tmp\venvs\venv1\Scripts\activate`



```
C:\tmp\venvs\venv1\Scripts>activate  
(venv1) C:\tmp\venvs\venv1\Scripts>
```

If this appears, you are working in the newly created virtual environment

- By activating the virtual environment, we switched from the system-wide Python environment to a newly created one (venv1).

Virtual Environments & pip

Please note: The exercises won't work if your only Python installation is based on Anaconda!

■ Using a virtual environment

- Let us start a Python shell in this virtual environment and repeat the random number generation to find out where `numpy` is located.
 - Execute `python`
 - Enter `import numpy`

■ What happened?

```
>>> import numpy
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'numpy'
>>> _
```

- `numpy` is not installed in our virtual environment. Only a few basic site-packages (i.e. external modules) were installed when we created the virtual environment.

Virtual Environments & pip

Please note: The exercises won't work if your only Python installation is based on Anaconda!

- Using a virtual environment (cont'd)
 - Close the Python interpreter by entering `exit()`
 - Enter `pip install numpy`

```
(venv1) C:\tmp\venvs\venv1\Scripts>pip install numpy
Collecting numpy
  Downloading numpy-1.20.2-cp39-cp39-win_amd64.whl (13.7 MB)
    |████████████████████| 13.7 MB 6.4 MB/s
Installing collected packages: numpy
Successfully installed numpy-1.20.2
```

- Start a new Python interpreter and try again to `import numpy`
 - It should work this time.
- Check the location of numpy:

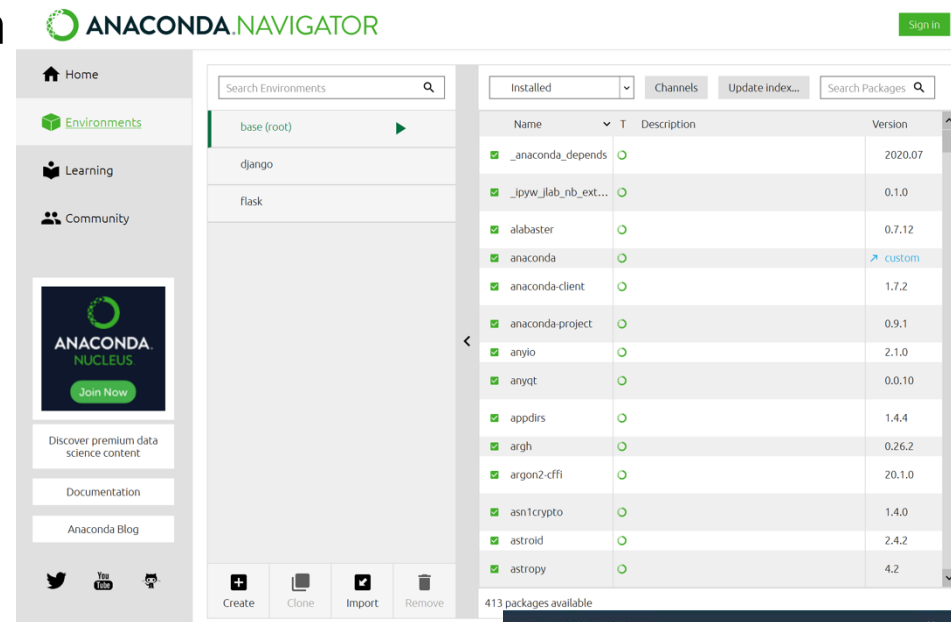
```
(venv1) C:\tmp\venvs\venv1\Scripts>python
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec 7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> numpy
<module 'numpy' from 'C:\\tmp\\venvs\\venv1\\lib\\site-packages\\numpy\\__init__.py'>
```

 This should be the path to your virtual environment

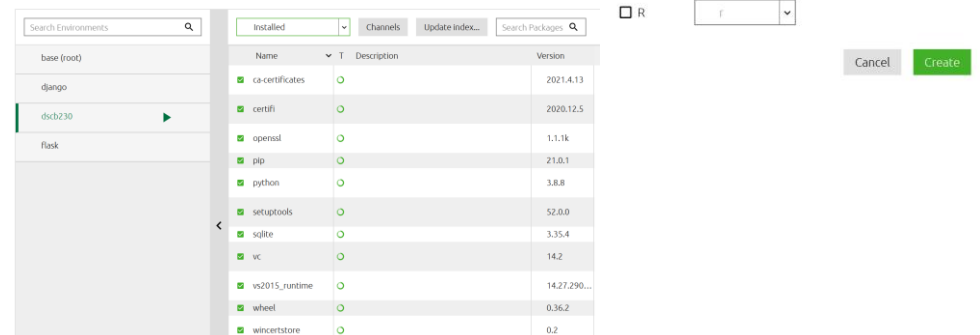
Conda Environments

- In case you are relying solely on Anaconda, please launch the Anaconda Navigator

- There, you can check the available conda environments (probably only base).
- You can also inspect the installed packages.

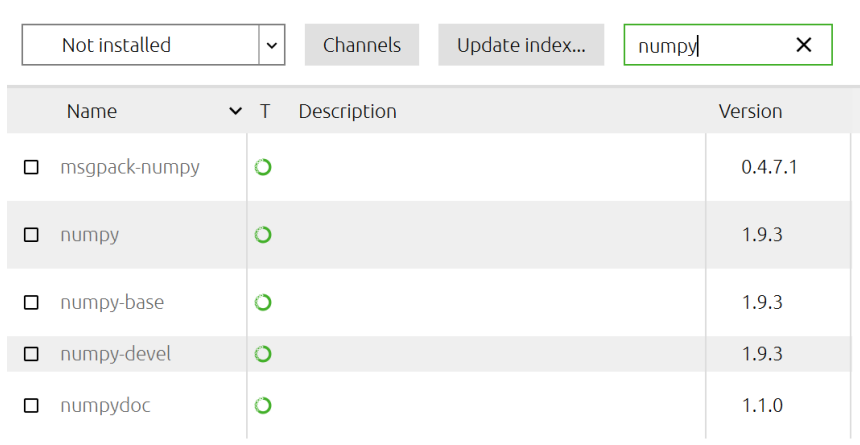


- You can create a new environment using the “Create” button.
 - The Python-version on which the environment should be based needs to be set.
- The newly created environment is activated.
 - Less packages are probably installed compared to “base”
 - A new folder should appear within Anaconda3\envs



Conda Environments & Package Management

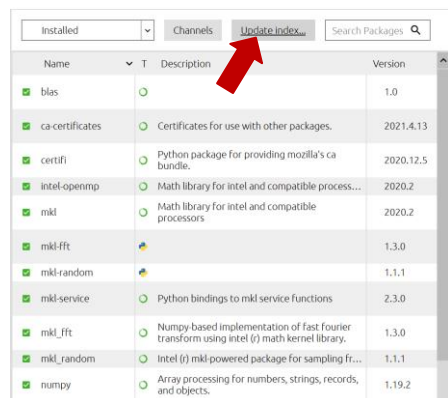
- Anaconda provides a convenient way of managing packages in the GUI via the conda package manager (this is different to pip!).



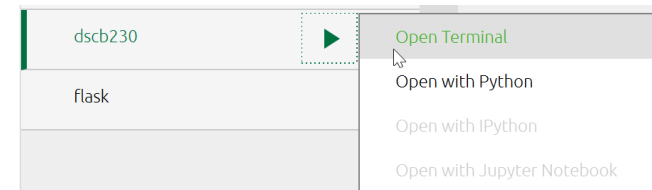
Name	T	Description	Version
☐ msgpack-numpy	○		0.4.7.1
☐ numpy	○		1.9.3
☐ numpy-base	○		1.9.3
☐ numpy-devel	○		1.9.3
☐ numpydoc	○		1.1.0

Please note: pip also works within conda environments

- You can also use conda from within a shell.
 - The installed packages should be visible in the UI



Name	T	Description	Version
☒ blas	○		1.0
☒ ca-certificates	○	Certificates for use with other packages.	2021.4.13
☒ certifi	○	Python package for providing mozilla's ca bundle.	2020.12.5
☒ intel-openmp	○	Math library for intel and compatible process...	2020.2
☒ mkl	○	Math library for intel and compatible processors	2020.2
☒ mkl_fft	○		1.3.0
☒ mkl_random	○		1.1.1
☒ mkl-service	○	Python bindings to mkl service functions	2.3.0
☒ mkl_fft	○	Numpy-based implementation of fast Fourier transform using intel (i) math kernel library.	1.3.0
☒ mkl_random	○	Intel (i) mkl powered package for sampling fr...	1.1.1
☒ numpy	○	Array processing for numbers, strings, records, and objects.	1.19.2



```
(dscb230) C:\Users\kupe0003>conda install numpy
Collecting package metadata (current_repodata.json): done
Solving environment: done

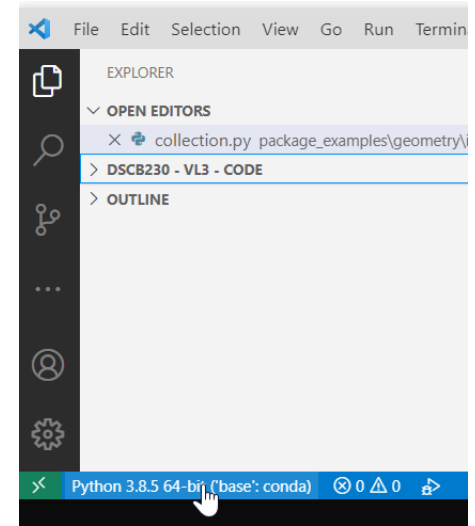
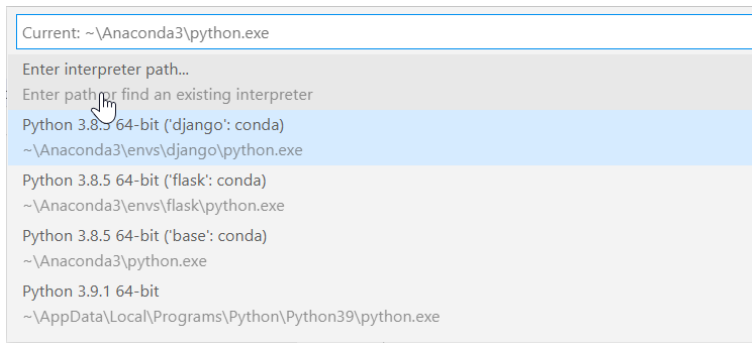
The following NEW packages will be INSTALLED:

blas                pkgs/main/win-64::blas-1.0-mkl
intel-openmp        pkgs/main/win-64::intel-openmp-2020.2-254
mkl                 pkgs/main/win-64::mkl-2020.2-256
mkl-service         pkgs/main/win-64::mkl-service-2.3.0-py38h196d8e1_0
mkl_fft             pkgs/main/win-64::mkl_fft-1.3.0-py38h46781fe_0
mkl_random          pkgs/main/win-64::mkl_random-1.1.1-py38h47e9c7a_0
numpy               pkgs/main/win-64::numpy-1.19.2-py38hdc3359_0
numpy-base          pkgs/main/win-64::numpy-base-1.19.2-py38ha3acd2a_0
six                 pkgs/main/win-64::six-1.15.0-py38haa95532_0

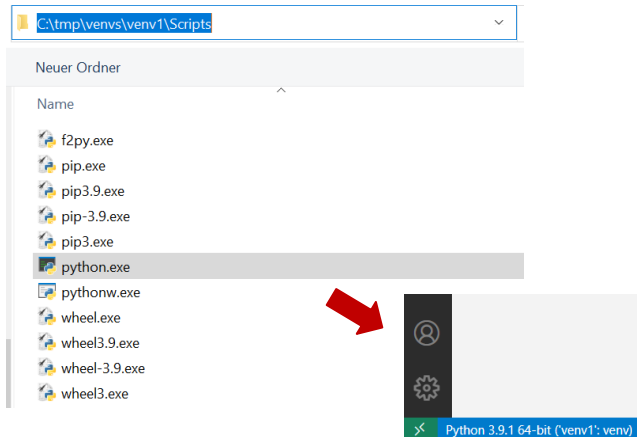
Proceed ([y]/n)?
```

Virtual / conda Environments & VSCode

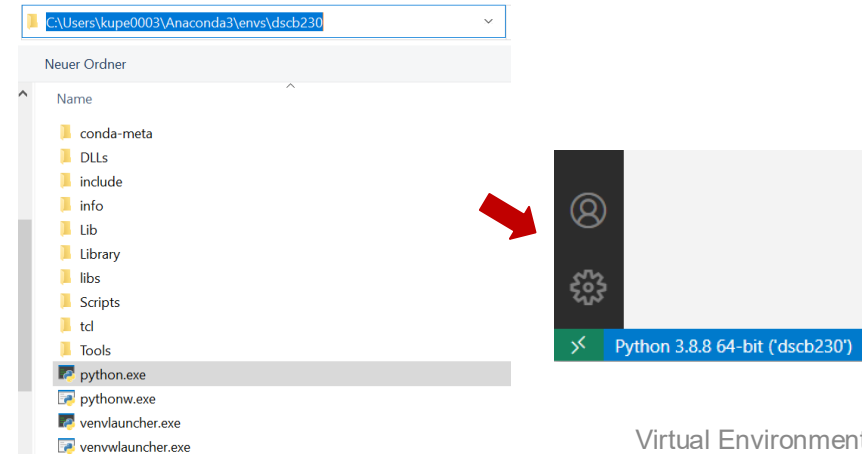
- We can switch the Python interpreter in VSCode
 - If your conda/virtual environment is not show, you'll need to add it.



- Select the respective python.exe within the venv/conda folder
virtualenv:



conda env:



Virtual / conda Environments & VSCode

- As a preparation for quickly installing the virtual environment on another machine (or container), we need to specify which packages are installed with which versions.
 - This is stored in a file called requirements.txt.
 - Within an activated virtual environment, we can write this file using the currently installed packages: `pip freeze > requirements.txt`
 - See https://pip.pypa.io/en/latest/user_guide/#requirements-files
 - Assuming that we also installed pandas via pip, our virtual environment's file could look like this:

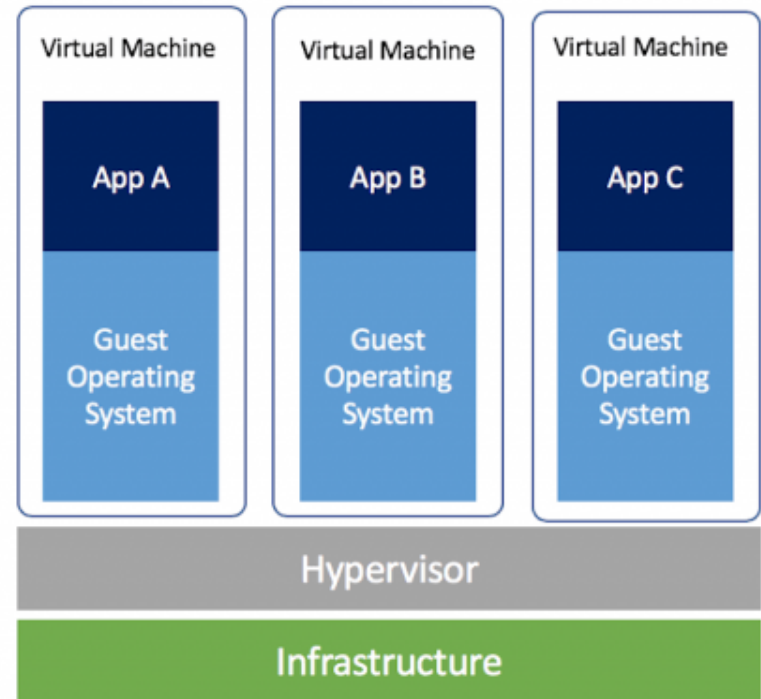
```
numpy==1.20.2
pandas==1.2.4
python-dateutil==2.8.1
pytz==2021.1
six==1.15.0
```
- Difference to install_requires:
 - While install_requires defines the dependencies for a single project, Requirements Files are often used to define the requirements for a complete Python environment.

Agenda

- ☐ Grundlagen der Modularisierung in Python
- ☐ Definition eigener Python-Packages
- ☐ Dependency Management
- ☐ Virtuelle Umgebungen (virtualenv & conda)
- ☒ Ausblick auf Containerisierung
- ☐ Zusammenfassung und Ausblick

Containerization – Fundamentals

- Virtual Machine (VM)
 - Isolates applications and allocates resources to run that application
 - VMs can be shared as images
 - Aren't dependent on the Host OS
 - Multiple VMs can be run simultaneously using a hypervisor



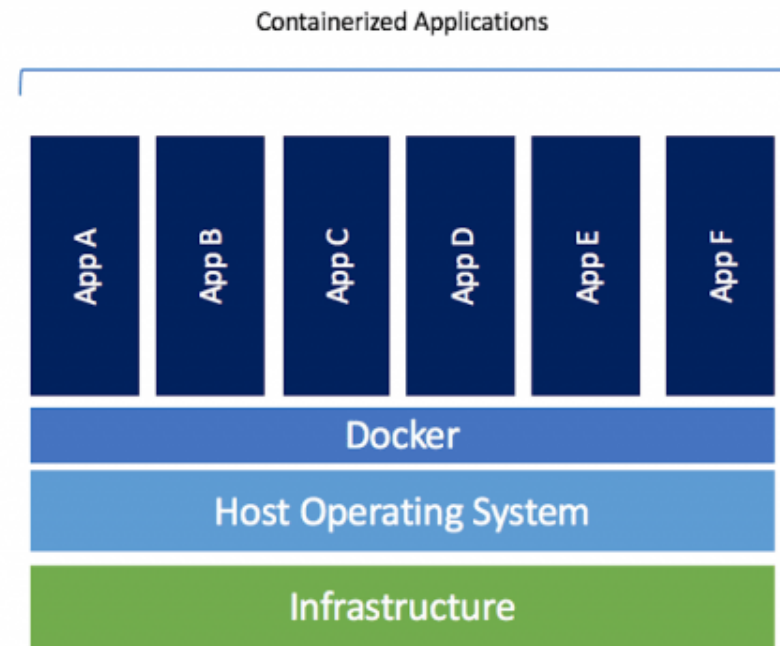
Containerization – Fundamentals

■ Docker Containers

- Standard unit of software
- Packages code and dependencies
- Can be shared as Docker Images
- Multiple containers can be run simultaneously
- Portable - Can be used with any OS
- Lightweight - Uses the host operating system
- Secure - Strong default isolation features

■ Why use containers?

- Develop applications that work on any OS
- Easy to share applications among teams
- Easy to scale across multiple servers
- Large applications can be broken into multiple containers - one per microservice
- Well-suited for Cloud Computing
- Big community and library of Docker Images (<https://hub.docker.com/>)





Exercise

- Untersucht den Code und erklärt was die Anwendung macht
 - <https://github.com/lukas-weiss/simpleapp>
- Startet einen Container auf eurem eigenen Rechner
 - <https://hub.docker.com/repository/docker/lweiss89/simpleapp/general>

```
docker run -dit -p 1337:1337 docker.io/lweiss/simpleapp
```

- Was könnt ihr beobachten?
-> ruft im Browser <http://localhost:1337/> auf



Exercise

- Untersucht den Code und erklärt was die Anwendung macht
 - <https://github.com/lukas-weiss/lecture-task-app>
- Startet einen Container auf eurem eigenen Rechner
 - <https://hub.docker.com/repository/docker/lweiss89/lecture-task-app/general>

```
docker compose up -d
```

- Was könnt ihr beobachten?
- Startet die Container neu und beobachtet was passiert

Containerization – Summary

- Containers are the basis of most current software projects.
- The architectural style of “microservices” can be implemented using containers.
 - We won’t go into detail here.
- Orchestration tools like Kubernetes build upon containers in order to build and manage complex application architectures.
 - Open source tool for **managing containerized applications** (e.g. provisioning of a website, dask cluster, spark cluster, ...).
 - It **relies on containers** as the basis for deploying the parts of an application (e.g. apache webserver, nginx load balancer, mysql database, Jupyter server, dask scheduler, dask worker nodes, etc.)
 - We won’t go into further detail here.

Agenda

- ☐ Grundlagen der Modularisierung in Python
- ☐ Definition eigener Python-Packages
- ☐ Dependency Management
- ☐ Virtuelle Umgebungen (virtualenv & conda)
- ☐ Ausblick auf Containerisierung
- ☒ Zusammenfassung und Ausblick

Zusammenfassung und Ausblick



- Sie sollten nun die grundlegenden Konzepte und Ansätze zur Modularisierung in Softwarearchitekturen anhand von Packages wiedergeben können.
 - Fokus: Python
- Sie sind nun in der Lage, eigene Python-Packages zu programmieren.
- Sie kennen den Nutzen virtueller Umgebungen (virtualenv & conda) und können eigene Umgebungen starten und darin Pakete installieren.
- Anhand eines Ausblicks sollten Sie das große Potential der Containerisierung erkannt haben und sind in der Lage, eigene Container-Images (grundlegend) zu definieren und auf einer (virtuellen) Maschine, die Docker installiert hat, laufen zu lassen.

Vielen Dank für Ihre Aufmerksamkeit!

